Exp No:14 Experiment to understand feature scaling.

AIM:

To understand and implement feature scaling techniques on a given dataset.

Description:

Understand the importance of feature scaling.

CODE:

```
import numpy as np
import pandas as pd
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("feature_scaling_dataset.csv")
df
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
df
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
features
from sklearn.impute import SimpleImputer
age_imputer = SimpleImputer(strategy="mean")
salary_imputer = SimpleImputer(strategy="mean")
age_imputer.fit(features[:, [1]])
salary_imputer.fit(features[:, [2]])
features[:, [1]] = age_imputer.transform(features[:, [1]])
features[:, [2]] = salary_imputer.transform(features[:, [2]])
features
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
```

```
Country
final_set = np.concatenate(
   (Country, features[:, [1, 2]]),
   axis=1
)
final_set
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
sc.fit(final_set)
feat_standard_scaler = sc.transform(final_set)
feat_standard_scaler
from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)
feat_minmax_scaler = mms.transform(final_set)
feat_minmax_scaler
```

OUTPUT:

1. Original Dataset

	Country	Age	Salary	Purchased
0	France	42.0	68000.0	No
1	Spain	29.0	51000.0	Yes
2	Germany	31.0	57000.0	No
3	Spain	36.0	64000.0	Yes
4	Germany	NaN	NaN	No
5	France	34.0	61000.0	Yes
6	Spain	NaN	55000.0	No
7	France	46.0	82000.0	Yes
8	Germany	51.0	76000.0	No
9	France	39.0	70000.0	Yes

2. Features After Missing Values Are Filled

```
array([['France', 42.0, 68000.0],
       ['Spain', 29.0, 51000.0],
       ['Germany', 31.0, 57000.0],
       ['Spain', 36.0, 64000.0],
       ['Germany', 38.5, 64888.88888888889],
       ['France', 34.0, 61000.0],
       ['Spain', 38.5, 55000.0],
       ['France', 46.0, 82000.0],
       ['Germany', 51.0, 76000.0],
       ['France', 39.0, 70000.0]], dtype=object)
```

3. One-Hot Encoding

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

4. Final Dataset Before Scaling

```
array([[1.0, 0.0, 0.0, 42.0, 68000.0],
       [0.0, 0.0, 1.0, 29.0, 51000.0],
       [0.0, 1.0, 0.0, 31.0, 57000.0],
       [0.0, 0.0, 1.0, 36.0, 64000.0],
       [0.0, 1.0, 0.0, 38.5, 64888.88888888889],
       [1.0, 0.0, 0.0, 34.0, 61000.0],
       [0.0, 0.0, 1.0, 38.5, 55000.0],
```

[1.0, 0.0, 0.0, 46.0, 82000.0],
[0.0, 1.0, 0.0, 51.0, 76000.0],
[1.0, 0.0, 0.0, 39.0, 70000.0]], dtype=object)

5. StandardScaler Output

array([[1.22474487, -0.65465367, -0.65465367, 0.5547873 , 0.34337869],
[-0.81649658, -0.65465367, 1.52752523, -1.50585123, -1.53294056],
[-0.81649658, 1.52752523, -0.65465367, -1.18882992, -0.87071024],
[-0.81649658, -0.65465367, 1.52752523, -0.39627664, -0.0981082],
[-0.81649658, 1.52752523, -0.65465367, 0.   , 0.   ],
[1.22474487, -0.65465367, -0.65465367, -0.71329795, -0.42922336],
[-0.81649658, -0.65465367, 1.52752523, 0.   , -1.09145368],
[1.22474487, -0.65465367, -0.65465367, 1.18882992, 1.88858277],
[-0.81649658, 1.52752523, -0.65465367, 1.9813832 , 1.22635245],
[1.22474487, -0.65465367, -0.65465367, 0.07925533, 0.56412213]])

6. MinMaxScaler Output

array([[1.   , 0.   , 0.   , 0.59090909, 0.5483871],
[0.   , 0.   , 1.   , 0.   , 0.   ],
[0.   , 1.   , 0.   , 0.09090909, 0.19354839],
[0.   , 0.   , 1.   , 0.31818182, 0.41935484],
[0.   , 1.   , 0.   , 0.43181818, 0.44802867],
[1.   , 0.   , 0.   , 0.22727273, 0.32258065],
[0.   , 0.   , 1.   , 0.43181818, 0.12903226],
[1.   , 0.   , 0.   , 0.77272727, 1.   ],
[0.   , 1.   , 0.   , 1.   , 0.80645161],
[1.   , 0.   , 0.   , 0.45454545, 0.61290323]])

RESULT:

The dataset was successfully preprocessed and scaled using standard scaler and minmax.